

Relatório Individual de Produção

Projeto Pokédex — Etapa 2 (Final)

Aluno: Luís Leão

Disciplina / Professor: Akira _____

Grupo: G10

Repositório do grupo (GitHub): <https://github.com/poo-ec-2026-1/g10>

Data: julho de 2026

1. Atribuição de cargo e tarefas

Cargo / responsabilidade atribuída:

Sei responsável pela camada de Banco de Dados do projeto, mesma atribuição da Etapa 1. Nesta Etapa 2, a responsabilidade se ampliou para atender novas necessidades da interface: além de manter e evoluir a camada já entregue, precisei implementar dois novos repositórios (Equipe e MembroTime), solicitados pelo colega responsável pela interface para permitir a persistência do time montado pelo usuário.

Tarefas atribuídas a priori:

- Manter a camada de banco de dados entregue na Etapa 1 (conexão, repositórios existentes, rotina de carga).
- Implementar dois novos repositórios (EquipeRepository e MembroTimeRepository) para a persistência do time do usuário.
- Padronizar esses novos repositórios seguindo o mesmo padrão dos anteriores (padrão Repository do tutorial).
- Preparar e gravar a apresentação em vídeo da minha parte do projeto.

O que foi exercido na prática:

Exerci todas as responsabilidades atribuídas. Além disso, atuei como integrador entre a camada de banco de dados e as demandas da interface: quando o colega responsável pela interface enviou uma primeira versão dos repositórios em um padrão diferente do adotado no projeto (método salvar em vez de create, recebendo ConnectionSource em vez de Database), identifiquei a inconsistência e reescrevi ambos no padrão unificado do projeto, garantindo que os repositórios continuassem coerentes com os demais.

2. Contribuição de acordo com a atribuição

O que foi cumprido:

- Implementação do EquipeRepository, seguindo o padrão Repository do tutorial: dao estático, método setDatabase para preparar DAO e tabela, e operações create, loadFromId, loadAll, update, delete e deleteById.
- Implementação do MembroTimeRepository, também no padrão Repository, com o mesmo CRUD completo e um método adicional loadByEquipe(equipe), que utiliza QueryBuilder do ORMLite para retornar os membros de uma equipe específica.
- Padronização dos dois novos repositórios em relação aos entregues na Etapa 1: mesmo estilo de construtor (recebendo Database), mesma nomenclatura de métodos (create, loadFromId, loadAll), mesma estrutura de tratamento de erros.
- Ampliação da documentação inline (Javadoc) dos métodos principais nos repositórios PokemonRepository, EquipeRepository, MembroTimeRepository e na

classe Database, endereçando o ponto "faltou documentação" apontado no feedback da Etapa 1.

- Manutenção da camada de banco entregue na Etapa 1 (Database, PokemonRepository, GolpeRepository, NatureRepository, TipoEfetividadeRepository e PopularBanco).
- Preparação e gravação da apresentação em vídeo da minha parte do projeto, com explicação dos arquivos e das linhas mais relevantes de cada um.

Três commits mais relevantes:

1) <https://github.com/poo-ec-2026-1/g10/commit/b7353e85ee5f75d89a487e6c3b7515dbfb932ac7> —

Adiciona documentação Javadoc no PokemonRepository, cobrindo os métodos create, loadFromId, loadAll, loadByName, loadByType e fraquezasDe (o mais complexo, que combina os tipos com a tabela de efetividade).

2) <https://github.com/poo-ec-2026-1/g10/commit/a870860118d0feda0a1e72b84362fe9c57359c4c> —

Adiciona documentação Javadoc no MembroTimeRepository, descrevendo o CRUD completo e o uso do QueryBuilder no método loadByEquipe.

3) <https://github.com/poo-ec-2026-1/g10/commit/7e4bdf4882d82591049edfef9ff1714f2616aa9> —

Adiciona documentação Javadoc no Database, descrevendo o construtor, o getConnection (com reuso da conexão) e o close.

Resposta ao feedback da Etapa 1:

Na Etapa 1 recebi retorno positivo sobre a implementação da carga de banco e o uso de LIKE, além do reconhecimento pela criação das estruturas de Efetividade e Natures. O ponto identificado como oportunidade de melhoria foi a documentação. Nesta Etapa 2, mantive a camada consistente e ampliei a contribuição com os dois novos repositórios sob demanda da interface, buscando responder ao ponto de "volume adequado, mas poderia ter desenvolvido um pouco mais" indicado no feedback.

Principais dificuldades:

- Padronizar os repositórios enviados por um colega, mantendo o padrão único do projeto sem retrabalho para os demais integrantes.
- Coordenar as fronteiras entre a minha camada e a etapa de Modelagem, para garantir que os repositórios operem sobre entidades compatíveis.

3. Contribuição além do atribuído

Além do que foi atribuído estritamente aos repositórios novos, contribuí com a equipe nos seguintes pontos:

- Padronização técnica: identifiquei e corriji a inconsistência de padrão nos repositórios enviados por um colega, garantindo coerência arquitetural no projeto.
- Ponte técnica com a interface: mantive comunicação direta com o responsável pela interface, ajustando os métodos dos repositórios ao que a interface efetivamente precisa consumir (buscar membros por equipe, atualizar e excluir times).
- Apresentação em vídeo: preparei roteiro e gravei a apresentação da minha parte com explicação linha a linha das partes mais relevantes do código, servindo como material de referência para o grupo.

Commits / documentos adicionais relevantes:

• <https://github.com/poo-ec-2026-1/g10/commit/3247b34773df46867870f4544bef306dfb6de132> — Commit-base da camada de banco (Etapa 1), sobre o qual esta etapa evoluiu, mantido como referência técnica.

4. Considerações gerais

O que aprendi:

- Como manter um padrão arquitetural único quando o projeto cresce e novos componentes são adicionados por diferentes integrantes.
- Aplicação prática do QueryBuilder do ORMLite para consultas com filtro, além dos métodos simples (queryForId, queryForAll) usados até então.
- A importância de definir claramente as fronteiras entre camadas em um trabalho em grupo: quando a modelagem, a persistência e a interface estão em responsabilidades diferentes, alinhar interfaces técnicas cedo evita retrabalho.

Conclusão:

Na Etapa 2 mantive a camada de banco entregue anteriormente e a ampliei com dois novos repositórios (Equipe e MembroTime) para atender à necessidade de persistência do time do usuário. O foco desta etapa foi a coerência arquitetural. Considero a camada de dados estabilizada e pronta para uso na integração final.